

Assignment No - 2

Q1] Analyze the behaviour of selection Sort for best-case, average-case input arrangements. Compare the number of comparisons and swaps performed in each case and justify its overall time complexity using asymptotic notation.

Ans Selection Sort :-

Selection Sort is a comparison-based sorting algorithm. It divides the array into two parts:

1. Sorted ^{portion} position - Elements that have already been placed in their correct positions.
2. Unsorted Portion - Elements that still need to be sorted

In every pass, Selection Sort finds the smallest element from the unsorted portion and places it at the beginning of that portion.

Algorithm:

For an array of n element :-

- Step 1: Start from the first position.
- Step 2: Assume the current element is the minimum.
- Step 3: Compare it with all remaining elements.
- Step 4: Find the smallest element.
- Step 5: Swap the smallest element with the current element.
- Step 6: Repeat until the entire array is sorted.

Example :-

Consider :

[64, 25, 12, 22, 11]

Pass 1 : Minimum = 1

Swap 64 and 11 : [11, 25, 12, 22, 64]

Pass 2 : Minimum = 12

Swap 25 and 12 : [11, 12, 25, 22, 64]

Pass 3 : Minimum = 22

Swap 25 and 22 : [11, 12, 22, 25, 64]

Pass 4 : Minimum = 25

No actual change required.

Final : [11, 12, 22, 25, 64]

Number of Comparisons -

Selection Sort performs almost the same number of comparisons regardless of the input arrangement.

For n elements :

- First pass $\rightarrow n-1$ comparisons
- Second pass $\rightarrow n-2$
- Third pass $\rightarrow n-3$
-
- Last pass $\rightarrow 1$

Therefore,

$$C(n) = (n-1) + (n-2) + \dots + 2 + 1$$

Using the sum of the first $n-1$ natural numbers:

$$C(n) = \frac{n(n-1)}{2}$$

Therefore,

$$C(n) = \frac{n^2 - n}{2}$$

Hence,

$$C(n) = \theta(n^2)$$

• Best-Case Analysis -

The best case occurs when the array is already sorted.

Example -

[10, 20, 30, 40, 50]

Even though the array is already sorted, selection sort still searches the entire unsorted portion to determine the minimum element.

Therefore,

$$\text{Comparisons} \therefore \frac{n(n-1)}{2}$$

Swaps :- If the implementation performs a swap only when the minimum element is different from the current element, then : 0 swaps for an already sorted array.

Thus,

$$T_{\text{best}}(n) = \theta(n^2)$$

• Average - Case Analysis -

In the average case, the elements are in a random arrangement. Selection Sort still scans the entire unsorted portion during every pass.

Therefore, the number of comparisons remains:

$$\frac{n(n-1)}{2}$$

The number of swaps is at most $n-1$. On average, approximately a constant fraction of the positions require actual exchange, but the important point is that the number of swaps is $O(n)$.

Therefore:

$$T_{\text{average}}(n) = \Theta(n^2)$$

• Worst - Case Analysis -

The worst case occurs, when the array is arranged in a way that causes maximum rearrangement.

Example:

[50, 40, 30, 20, 10]

Selection sort still performs:

$$\frac{n(n-1)}{2} \text{ comparisons}$$

At most one swap is performed during each pass, giving:

$n-1$ swaps

Therefore,

$$T_{\text{worst}}(n) = \Theta(n^2)$$

Q2] Demonstrate the time-complexity analysis of Dijkstra's Algorithm with (a) adjacency matrix and array and (b) adjacency list and min-heap, and compare their efficiency.

Ans Dijkstra's Algorithm -

Dijkstra's Algorithm is a single-source shortest path algorithm used to find the shortest distance from a source vertex to every other vertex in a weighted graph.

It works when the graph has non-negative edge weights.

Let:

- V = number of vertices
- E = number of edges

The efficiency of Dijkstra's Algorithm depends significantly on the data structure used to represent the graph and the priority queue used to select the next minimum-distance vertex.

(a) Adjacency Matrix and Array

In this Implementation:

- The graph is represented using an adjacency matrix.
- An Array is used to maintain the shortest distance.
- At every step, the unvisited vertex having the minimum distance is selected.

Steps:-

Step 1 - Initialize the distance of the source vertex as 0.

Step 2 - Initialize the distance of all other vertices as infinity.

Step 3 - Select the unvisited vertex with the minimum distance.

Step 4 - Mark it as visited

Step 5 - Examine its adjacent vertices

Step 6 - Update their distance if a shorter path is found.

Step 7 - Repeat until all vertices are processed.

Finding the Minimum Vertex -

There are v vertices, and finding the minimum-distance unvisited vertex using an array requires:

$$O(v)$$

This operation is performed v times:

$$O(v) \times O(v) = O(v^2)$$

Processing Edges

With an adjacency matrix, for each selected vertex, we scan the entire row of matrix.

This takes:

$$O(v) \text{ per vertex}$$

For all vertices:

$$O(v^2)$$

Therefore, total complexity is

$$O(v^2)$$

The space complexity of the adjacency matrix is

$$O(v^2)$$

This implementation is particularly suitable for dense graphs, where the number of edges is close to:

$$E = v^2$$

(b) Adjacency List and Min-Heap

In this implementation:

- The graph is represented using an adjacency list.
- A min-heap/priority queue is used to efficiently obtain the vertex having the smallest tentative distance.

The min-heap allows us to extract the minimum-distance vertex efficiently.

Heap Operations

The main operations are:

Extract-Min :-

$$O(\log V)$$

Decrease-key / Updating priority :-

$$O(\log V)$$

Each vertex is processed through the priority queue, and the edges are relaxed.

Vertex Processing

For V vertices

$$O(V \log V)$$

Edge Processing

Each edge is considered during relaxation.

For E edges:

$$O(E \log V)$$

Therefore:

$$O(V \log V + E \log V)$$

$$O((V+E) \log V)$$

Space Complexity

The adjacency list requires:

$O(V+E)$ space

The min-heap requires additional space, generally:

$O(V)$

Thus overall space complexity is

$O(V+E)$

Comparison :-

Feature	Adjacency Matrix+Array	Adjacency List + Min-Heap
Graph representation	$O(V^2)$ Adjacency Matrix	$O(\log V)$ Adjacency List
Minimum selection	$O(V)$	$O(\log V)$
Edge processing	$O(V^2)$ overall	$O(E \log V)$
Time Complexity	$O(V^2)$	$O((E+V) \log V)$
Space Complexity	$O(V^2)$	$O(E+V)$
Best suited for	Dense graphs	Sparse graphs
Implementation	Simpler	More complex
Efficiency for sparse graphs	Lower	Higher

Q3] A Railway Reservation system stores available seat numbers in sorted order. Analyze whether Binary Search or Linear Search is more appropriate for locating a particular seat number when the number of available seats is very large. Justify your choice based on the number of comparisons and time complexity.

Ans The railway reservation system stores available seat numbers in sorted order.

For example:
[101, 105, 110, 115, 120, 125, 130, 135]

Suppose we want to search for seat number 125.

Two possible searching techniques are:

1. Linear Search
2. Binary Search

Since the seat numbers are already sorted, Binary is more appropriate, especially when the number of seats is very large.

Linear Search:

Linear Search checks each element one by one from the beginning.

For example:

[101, 105, 110, 115, 120, 125, 130, 135]

To Find 125:

1. Compare with 101
2. Compare with 105
3. Compare with 110
4. Compare with 115
5. Compare with 120
6. Compare with 125 → Found

Therefore, 6 comparisons are required in this particular case

Time Complexity -

Best Case: The required seat is the first element.
 $O(1)$

Average Case: Approximately half elements may need to be checked
 $O(n)$

Worst Case: The seat is in the last position or is not present
 $O(n)$

Thus,

$T_{\text{linear}}(n) = O(n)$

Binary Search

Binary Search takes advantage of the fact that the seat numbers are already sorted

It repeatedly divides the search range into two halves.

For Example:

[101, 105, 110, 115, 120, 125, 130, 135]

Middle Element: 115

Since $125 > 115$

we ignore the left half

Remaining:

[120, 125, 130, 135]

Middle element: 130

Since $125 < 130$

we search the left half

Remaining:

[120, 125]

Now we can find 125

The search space is reduced approximately by half at every step.

Time Complexity of Binary Search:

After one comparison:

$$n \rightarrow \frac{n}{2}$$

After two comparisons:

$$n \rightarrow \frac{n}{4}$$

After k comparisons

$$\frac{n}{2^k}$$

When only one element remains

$$\frac{n}{2^k} = 1$$

Therefore :

$$2^k = n$$

Taking logarithm

$$k = \log_2 n$$

Hence,

$$\text{Binary}(n) = O(\log n)$$

Binary Search is the better choice for the railway reservation system because:

1. The seat numbers are already sorted
2. The no. of available seats can be very large
3. Binary Search reduces the search space by half after every comparison

4. Its worst-case time complexity is $O(\log n)$

Compared with linear Search

$$O(n)$$

Therefore, Binary Search is preferred over Linear Search. because it requires approximately $\log_2(n)$ comparisons instead of upto n comparisons.